CCPS LAB EXAM

Answered Practical Cheatsheet — Common Exam-Style Questions

Includes the original quick revision topics plus one worked example from each major area in your question bank: Node-RED/dashboard, Arduino sensor logic, TCP sockets, Raspberry Pi interfacing, CoAP, JSON validation, and integrated CPS flow.

Exam sequence: identify inputs → connect/configure hardware → write code or build flow → run/deploy → show output → state result.

Important: Replace example IP addresses with the actual Raspberry Pi server IP. Pin numbering below uses BCM numbering unless stated otherwise. For an HC-SR04 ultrasonic sensor, do not connect its 5 V Echo pin directly to Raspberry Pi GPIO; use a voltage divider or level shifter.

1. NODE-RED — Three-Level Temperature Monitoring

Question: Classify temperature as Below 25°C = Comfortable; 25–30°C = Normal; Above 30°C = Hot. Display the status on a dashboard.

Exact flow to build (quick simulation without Arduino):

1. Drag an **Inject** node. Double-click it → set Payload type to **number**, value **24** (you can change it to 28 or 32 to test). Set it to inject once when clicked. Click Done.
2. Drag a **Function** node. Connect **Inject** → **Function**. Double-click Function and paste the code below. Click Done.
3. Drag a **Debug** node and connect **Function** → **Debug**. Set Debug to show **msg.payload**.
4. For a dashboard display, install **node-red-dashboard** from Menu ■ → Manage palette → Install (classic dashboard, if it is not already installed). Add a **ui_text** node and connect **Function** → **ui_text**. Set its Group/Tab, label it “Temperature Status”, and set Value format to {{msg.payload}}.
5. Click **Deploy**, then click the Inject button. Open http://localhost:1880/ui to see the dashboard.

```
// Function node: temperature in msg.payload
let t = Number(msg.payload);

if (Number.isNaN(t)) {
    msg.payload = "Sensor Failure";
} else if (t < 25) {
    msg.payload = "Comfortable";
} else if (t <= 30) {
    msg.payload = "Normal";
} else {
    msg.payload = "Hot";
}
return msg;
```

Expected tests: input 24 → Comfortable; input 28 → Normal; input 32 → Hot. If you use a Gauge, send a numeric temperature to it; send the text status to a separate ui_text widget.

2. NODE-RED — Occupancy Alert (Question 1)

Question: If ultrasonic distance is below 25 cm, show “Student Detected”; otherwise show “No Student”. Temperature and humidity may be displayed separately.

Flow: Arduino serial JSON → **serial in** → **json** → **Function** → **ui_text**. To test without hardware, use **Inject (JSON)** instead of serial in. Connect the nodes in that order. In the Function node, use:

```
// Incoming object example: { "distance": 18,
 // "temperature": 27, "humidity": 60 }
let d = Number(msg.payload.distance);
msg.payload = (Number.isFinite(d) && d < 25)
    ? "Student Detected"
    : "No Student";
return msg;
```

For a real serial connection, install/select the serial port and baud rate matching the Arduino sketch. The Arduino must send one complete JSON object per line, followed by a newline. Add separate Change/Function branches for temperature and humidity widgets.

3. ARDUINO + NODE-RED — Temperature/Humidity Alert

Question: Alert when temperature > 30°C OR humidity > 75%.

Arduino should send JSON like {"temperature":31,"humidity":60} on Serial, one object per line. In Node-RED use **serial in** → **json** → **Function** → **ui_text**. Set serial baud rate to match the Arduino code (for example 9600).

```
let t = Number(msg.payload.temperature);
let h = Number(msg.payload.humidity);

if (!Number.isFinite(t) || !Number.isFinite(h)) {
    msg.payload = "Sensor Failure";
} else if (t > 30 || h > 75) {
    msg.payload = "UNCOMFORTABLE";
} else {
    msg.payload = "Normal";
}
return msg;
```

Remember: OR means either condition is enough. If the question says temperature > 30 AND humidity > 70, use && instead of ||.

4. TCP SOCKET — Three-Level Distance Safety

Question: Client sends distance to a server; server classifies it as SAFE / WARNING / DANGER. The question bank uses port 5683.

Run order: Start server.py first; find the server Raspberry Pi IP using hostname -I; put that IP in client.py; run client.py on the client device. Both devices should be on the same network.

```
# server.py
import socket

HOST, PORT = "0.0.0.0", 5683
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind((HOST, PORT))
    s.listen(2)
    print("Waiting for client...")
    conn, addr = s.accept()
    with conn:
        print("Connected:", addr)
        while True:
            data = conn.recv(1024)
            if not data:
                break
            try:
                d = float(data.decode().strip())
                if d < 20:
                    status = "DANGER"
                elif d <= 50:
                    status = "WARNING"
                else:
                    status = "SAFE"
                print(f"Distance={d:.1f} cm -> {status}")
            except ValueError:
                print("Invalid distance packet")
```

```
# client.py (simulated sensor; replace with sensor reading)
import socket, time

SERVER_IP = "192.168.1.10"  # replace this
PORT = 5683
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((SERVER_IP, PORT))
    try:
        while True:
            distance = 35.0
            s.sendall(str(distance).encode())
            time.sleep(2)
    except KeyboardInterrupt:
        print("Client stopped")
```

Expected: 15 → DANGER; 35 → WARNING; 60 → SAFE. For the physical ultrasonic sensor, replace the simulated distance with the sensor's measured value.

5. RASPBERRY PI GPIO — Ultrasonic + LED

Question: LED ON when distance < 20 cm, otherwise OFF. This is the local hardware decision example.

Example BCM pins: TRIG=18, ECHO=24, LED=17. LED long leg → GPIO17; short leg → 330Ω resistor → GND. HC-SR04 Echo needs level shifting/voltage divider before Raspberry Pi GPIO.

```
import RPi.GPIO as GPIO
import time

TRIG, ECHO, LED = 18, 24, 17
GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)
GPIO.setup(LED, GPIO.OUT)

def read_distance():
    GPIO.output(TRIG, False)
    time.sleep(0.05)
    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)

    timeout = time.time() + 0.04
    while GPIO.input(ECHO) == 0:
        if time.time() > timeout:
            return None
    start = time.time()

    timeout = time.time() + 0.04
    while GPIO.input(ECHO) == 1:
        if time.time() > timeout:
            return None
    end = time.time()
    return round((end - start) * 17150, 2)

try:
    while True:
        d = read_distance()
        if d is None:
            GPIO.output(LED, GPIO.LOW)
            print("ERROR_TIMEOUT")
        else:
            print("Distance:", d, "cm")
            GPIO.output(LED, GPIO.HIGH if d < 20 else GPIO.LOW)
        time.sleep(2)
except KeyboardInterrupt:
    pass
finally:
    GPIO.cleanup()
```

Formula: distance (cm) ≈ echo duration (seconds) × 17150. The timeout checks prevent the program from waiting forever if there is no echo.

6. COAP — Multiple Sensor Resources (Concept + Minimal Pattern)

Question: Create /temperature, /humidity and /light resources; each request returns only its corresponding value. Exact CoAP library APIs depend on the package used in your lab, so keep the resource mapping consistent with your existing lab code.

```
# Resource logic/pseudocode to adapt to your lab's CoAP library
latest = {
    "temperature": 28.5,
    "humidity": 62,
    "light": "DARK"
}

# GET /temperature  -> return str(latest["temperature"])
# GET /humidity     -> return str(latest["humidity"])
# GET /light        -> return str(latest["light"])
```

For threshold-based response: if temperature > 30 return HIGH, otherwise return NORMAL. For /light, return LIGHT_ON when light is DARK; return LIGHT_OFF when light is LIGHT. Use the exact server/resource syntax from the CoAP library in your lab record.

7. PYTHON — Ignore Invalid JSON + Add Timestamp

Question: Arduino sends JSON over serial; ignore corrupted lines and add a timestamp to valid sensor packets before storing them.

```
import json
from datetime import datetime

def handle_line(line):
    try:
        data = json.loads(line)  # invalid JSON raises an error
    except json.JSONDecodeError:
        print("Ignored invalid JSON:", line)
        return None

    data["timestamp"] = datetime.now().isoformat(
        timespec="seconds"
    )
    print("Valid packet:", data)
    return data

# Example:
handle_line('{"temperature":29,"humidity":65}')
handle_line('{bad json}')  # ignored
```

If using pySerial, read one line at a time with ser.readline().decode(errors='ignore').strip(), then pass it to handle_line(line). Install pySerial if needed with pip install pyserial.

8. MQTT — Basic Node-RED Sensor Flow

Question: Receive a sensor value from MQTT and display it on a dashboard.

Exact node order: mqtt in → json (only if payload is JSON text) → Function/Change (optional) → ui_text or gauge → Deploy.

1. Double-click **mqtt in**, add broker address/port and topic (must match publisher).
2. If the publisher sends JSON text, add a **json** node after mqtt in. If it sends a plain number, the JSON node may not be needed.
3. Connect to a **debug** node first to verify the incoming payload.
4. For a gauge, make sure msg.payload is a number. For text status, use **ui_text**.
5. Set dashboard Tab/Group, click Deploy, then open http://localhost:1880/ui.

If no messages arrive, check the broker connection, topic spelling, Wi-Fi/network, and whether a publisher is running.

9. INTEGRATED CPS — Temperature + Dark Room Decision

Question: Temperature > 30°C AND room is DARK → Critical Environment; only one condition true → Warning; neither → Normal.

```
# Node-RED Function node
let t = Number(msg.payload.temperature);
let light = String(msg.payload.light).toUpperCase();

if (!Number.isFinite(t) || !light) {
    msg.payload = "Sensor Failure";
} else {
    let hot = t > 30;
    let dark = light === "DARK";

    if (hot && dark) {
        msg.payload = "Critical Environment";
    } else if (hot || dark) {
        msg.payload = "Warning";
    } else {
        msg.payload = "Normal";
    }
}
return msg;
```

Flow: Serial In / MQTT In → JSON → Function → ui_text + Debug. Make sure the JSON field names are exactly temperature and light.

10. Quick Node-RED Setup + Common Viva

Start Node-RED on Windows: Open Command Prompt → run node -v and npm -v. If installed, run node-red. Keep the terminal open. Open http://localhost:1880 in the browser. If command is missing, install Node.js LTS first, then run npm install -g node-red.

Build a basic flow: drag Inject and Debug → connect output dot of Inject to input dot of Debug → Deploy → click Inject button → read the Debug sidebar.

Dashboard classic: Menu ■ → Manage palette → Install → search node-red-dashboard → Install. Add a ui_text/gauge node, assign Tab and Group, connect it, Deploy, open http://localhost:1880/ui.

Quick viva answers

What is a socket? An endpoint used by two programs to communicate over a network.
TCP vs UDP? TCP is connection-oriented and reliable; UDP is connectionless with lower overhead.
What does bind do? Assigns a local IP/port to the server socket.
What does listen/accept do? listen waits for incoming connections; accept accepts one client.
What does Deploy do in Node-RED? Applies the current flow configuration.
Why use a JSON node? Converts JSON text into a JavaScript object so fields can be accessed.
What is MQTT? A lightweight publish/subscribe messaging protocol.
What is CoAP? A lightweight REST-like protocol designed for constrained devices.
What does pinMode do in Arduino? Sets a pin as INPUT or OUTPUT.
Why validate sensor data? To prevent invalid readings from producing incorrect decisions or crashing code.

Exam tip: If the question asks for a dashboard, show the complete node order and assign each widget to a Tab/Group. If the question asks for hardware, mention the pin mapping and power/ground connections. Test at least one normal and one alert condition.